

FULL STACK DEVELOPMENT

FASE 5 | AULA 01 -

AS ARQUITETURAS DE MICROSERVIÇOS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON.....	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS.....	19
O QUE VOCÊ VIU NESTA AULA?	21
REFERÊNCIAS	22

EXEMPLO

O QUE VEM POR AÍ?

Nesta aula, exploraremos dois modelos arquiteturais fundamentais para o desenvolvimento de software: os monolitos e os microsserviços. A arquitetura de software é crucial para qualquer sistema bem-sucedido, pois define a estrutura, os componentes e as interações dentro do sistema. Escolher a arquitetura correta pode determinar o sucesso ou o fracasso de um projeto de software.

Existem diversas abordagens arquiteturais, mas duas das mais discutidas atualmente são os monolitos e os microsserviços. Monolitos são a abordagem tradicional, onde toda a aplicação é construída como uma única unidade indivisível. Nesta arquitetura, todas as funcionalidades e componentes estão integrados em um único código-base e são implantados juntos. Esta abordagem tem suas vantagens, especialmente em termos de simplicidade e facilidade de gerenciamento, mas também possui limitações significativas, como dificuldades de escalabilidade e manutenção.

Microsserviços, por outro lado, representam uma abordagem moderna. Eles dividem a aplicação em um conjunto de serviços pequenos e independentes, cada um executando uma função específica. Esses serviços comunicam-se entre si através de APIs bem definidas, promovendo a modularidade, a escalabilidade e a resiliência do sistema. No entanto, essa arquitetura também introduz desafios, como a complexidade de gestão e a necessidade de uma infraestrutura robusta.

Entender as diferenças entre essas duas abordagens é crucial para fazer escolhas informadas durante o desenvolvimento de software. Ao longo desta aula, vamos explorar em detalhes o que são monolitos e microsserviços, suas vantagens e desvantagens, e os cenários ideais para a aplicação de cada um. Mergulharemos nas características, estruturas e exemplos práticos para proporcionar uma compreensão abrangente dessas arquiteturas. Com esse conhecimento, você poderá se preparar para tomar decisões arquiteturais que atendam às necessidades específicas do seu projeto e promovam a eficiência, a escalabilidade e a resiliência do seu sistema.

HANDS ON

Nesta aula prática, você será apresentado(a) aos motivadores para escolher a arquitetura adequada para um sistema, destacando a importância de desenvolver aplicações que sejam não apenas eficientes e eficazes, mas também sustentáveis e adaptáveis a mudanças futuras. Vamos explorar como decidir entre arquiteturas monolíticas, microsserviços e monolitos modulares, considerando os fatores específicos do projeto, como tamanho do sistema, requisitos de escalabilidade e estrutura da equipe.



SAIBA MAIS

A arquitetura de software é fundamental para qualquer sistema bem-sucedido. Ela define a estrutura, os componentes e as interações dentro do sistema. É crucial entender as diferenças entre microsserviços e monolitos para fazer escolhas informadas durante o desenvolvimento de software.

O que são monolitos?

Começaremos pelos monolitos, a abordagem mais tradicional. Na arquitetura monolítica, construímos a aplicação como uma única unidade indivisível. Todos os componentes e funcionalidades da aplicação estão juntos em um único código-base e são implantados ao mesmo tempo.

Essa abordagem é bem comum e tem sido amplamente utilizada desde os primeiros dias do desenvolvimento de software. A grande vantagem aqui é a simplicidade, já que é possível ter tudo em um único lugar, o que facilita o desenvolvimento inicial e a implantação.

Estrutura de um monolito

A estrutura de uma aplicação monolítica geralmente segue um padrão bem definido. Vamos dar uma olhada nas principais camadas de uma aplicação monolítica típica:

Camada de apresentação

Esta é a camada que lida com a interface de usuário. Aqui, são gerados os formulários, páginas web, telas de aplicativos e outros elementos visuais com os quais o usuário interage.

Exemplo: em uma aplicação web, a camada de apresentação incluiria HTML, CSS e JavaScript que definem como os dados são exibidos ao usuário e como as interações (cliques, envio de formulários etc.) são tratadas.

Camada de lógica de negócios

Esta camada contém a lógica central da aplicação. Ela processa os dados, aplica regras de negócios e gerencia as operações que a aplicação deve realizar.

Exemplo: se você estiver desenvolvendo um sistema de e-commerce, a camada de lógica de negócios lidaria com cálculos de preços, validações de carrinho de compras, processamento de pedidos e processos similares.

Camada de acesso a dados

Esta camada é responsável por interagir com o banco de dados. Ela gerencia a persistência dos dados, lidando com consultas, atualizações, inserções e exclusões no banco de dados.

Exemplo: em um sistema de gerenciamento de inventário, a camada de acesso a dados incluiria os componentes que lidam com a leitura e gravação de informações sobre produtos, estoques e fornecedores.

Todas essas camadas estão interligadas em um único artefato de implantação. Isso significa que, quando você precisa fazer uma alteração em qualquer parte da aplicação, você deve modificar o código-base e reimplantar a aplicação inteira.

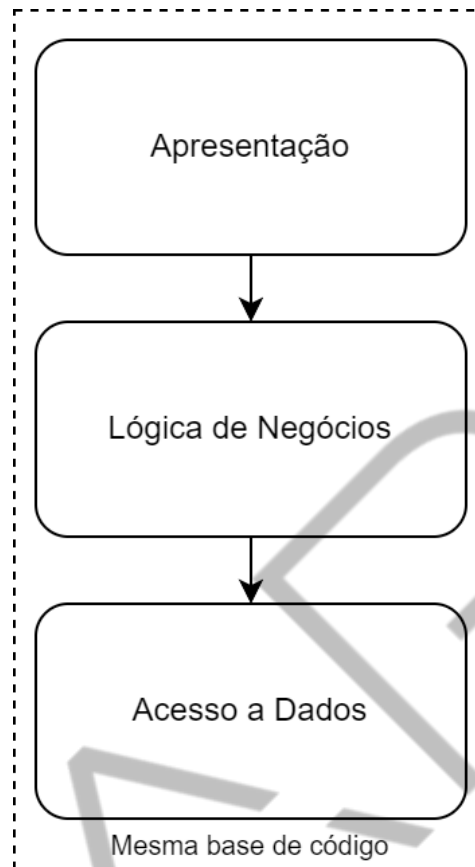


Figura 1 - Representação de um monolito

Fonte: elaborado pelo autor (2024)

Vantagens dos monolitos

A estrutura de um monolito é simples e fácil de entender, especialmente para equipes menores ou projetos iniciais. Como tudo está em um único lugar, os(as) desenvolvedores(as) podem se familiarizar com o código rapidamente. No cenário de uma pequena startup desenvolvendo seu primeiro produto, a equipe poderia trabalhar de forma mais coesa e eficiente com um único repositório de código.

Outra vantagem dos monolitos é referente à sua performance, pois a comunicação entre componentes é direta e rápida, já que tudo está dentro do mesmo processo. Isso pode resultar em melhor desempenho se compararmos com arquiteturas distribuídas. Uma aplicação interna de uma empresa que precisa processar grandes volumes de dados em tempo real pode se beneficiar da eficiência de um monolito, por exemplo.

Por fim, a última vantagem é o seu gerenciamento simplificado. Com um único código-base, o gerenciamento de versionamento e implantação é mais simples. São menos componentes para monitorar e gerenciar, o que significa menos sobrecarga administrativa. Uma equipe de TI pode facilmente gerenciar atualizações e patches em uma aplicação monolítica sem a necessidade de coordenar entre múltiplos serviços independentes.

Desvantagens dos Monolitos

Escalabilidade

Escalar uma aplicação monolítica geralmente envolve a replicação da aplicação inteira. Isso pode ser ineficiente, especialmente se apenas uma parte da aplicação requer mais recursos. Por exemplo, se a seção de relatórios está sob alta carga, você teria que escalar toda a aplicação, mesmo que outras partes não precisem de recursos adicionais.

Manutenção

À medida que a aplicação cresce, a manutenção se torna mais difícil e complexa. O código pode se tornar difícil de entender e modificar, especialmente se não houver uma boa separação de preocupações. Em uma grande aplicação legada, adicionar uma nova funcionalidade pode requerer a modificação de várias partes do sistema, aumentando o risco de introduzir bugs.

Implementação de mudanças

Qualquer mudança requer a reimplantação da aplicação inteira. Isso pode resultar em downtime e impactar a disponibilidade do sistema, o que é problemático para aplicações que precisam estar disponíveis 24/7 (24 horas por dia, 7 dias por semana). Uma loja on-line que precisa aplicar uma atualização de segurança durante um horário de pico pode enfrentar problemas se a aplicação inteira precisar ser reimplantada, por exemplo.

Exemplos de uso de monolitos

Sistemas legados

Muitos sistemas legados foram desenvolvidos usando uma arquitetura monolítica. Esses sistemas frequentemente continuam a operar dessa maneira porque a reescrita completa em uma nova arquitetura seria dispendiosa e arriscada.

Grandes sistemas de ERP (Enterprise Resource Planning), que gerenciam os processos internos de uma empresa, muitas vezes são monolíticos e continuam a ser utilizados devido à sua estabilidade e integridade de dados.

Pequenas startups

Startups e pequenas empresas frequentemente começam com uma arquitetura monolítica devido à sua simplicidade e rapidez de desenvolvimento. Esta abordagem permite que a equipe se concentre em construir e lançar o produto rapidamente.

Um exemplo prático disso seria um pequeno time desenvolvendo um novo aplicativo de agendamento de consultas. Eles podem começar com um monolito para acelerar o time-to-market.

Aplicações de menor escala

Aplicações que não esperam crescer significativamente ou que não possuem requisitos complexos de escalabilidade e manutenção podem se beneficiar da simplicidade de uma arquitetura monolítica. Uma aplicação de gerenciamento de eventos para uma pequena comunidade pode ser desenvolvida com um monolito, dado o escopo limitado e a simplicidade necessária.

É importante sempre considerar o contexto do seu projeto e as necessidades específicas antes de decidir qual arquitetura será utilizada.

O que são microsserviços?

A utilização de microsserviços é uma abordagem mais moderna, que divide a aplicação em um conjunto de serviços pequenos e independentes, cada um executando uma função específica, ao contrário da arquitetura monolítica, onde todas as funcionalidades estão agrupadas em um único código-base.

A ideia principal dos microserviços é promover a modularidade e a separação de preocupações. Cada microserviço é responsável por uma funcionalidade específica e se comunica com outros serviços através de APIs bem definidas. Isso permite que diferentes equipes trabalhem em diferentes serviços ao mesmo tempo, usando as tecnologias que melhor se adaptam às necessidades de cada serviço.

Por exemplo, em uma aplicação de e-commerce, você poderia ter:

- **Serviço de catálogo de produtos:** gerencia as informações dos produtos disponíveis para venda.
- **Serviço de carrinho de compras:** lida com a adição e remoção de produtos no carrinho de compras do usuário.
- **Serviço de pagamentos:** processa transações de pagamento.
- **Serviço de envio:** gerencia a logística de envio dos produtos comprados.

Cada um desses serviços pode ser desenvolvido, testado e implantado de forma independente. Se houver uma atualização necessária no serviço de pagamentos, você só precisa reimplantar esse serviço específico, sem afetar os outros. Além disso, eles podem se comunicar entre si, com camadas de apresentação e qualquer outra saída com as quais consigam se comunicar, como representado na figura 2:

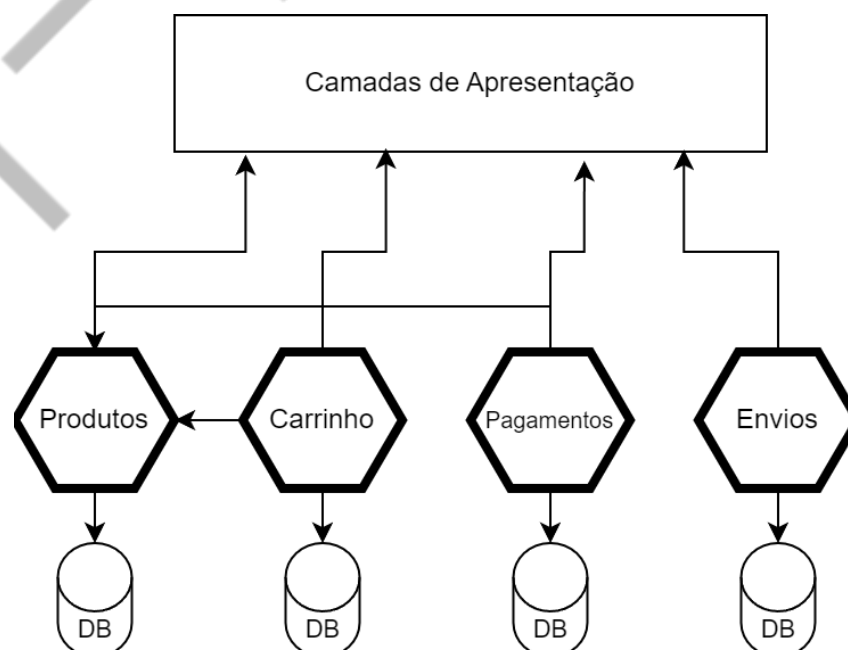


Figura 2 - Representação de microserviços interconectados

Fonte: elaborado pelo autor (2024)

Estrutura de microserviços

A arquitetura de microserviços segue um padrão modular:

- **Serviços independentes:** cada serviço é uma unidade funcional autônoma que realiza uma tarefa específica;
- **APIs de comunicação:** os serviços comunicam-se entre si através de APIs, geralmente utilizando protocolos HTTP/REST ou mensagens assíncronas;
- **Banco de dados descentralizados:** cada serviço pode ter seu próprio banco de dados, permitindo uma maior independência e especialização.

Detalhamento dos componentes

Serviços independentes

Cada microserviço é projetado para ser uma unidade funcional independente que pode ser desenvolvida, implantada e escalada de forma autônoma. Um cenário propício para esta arquitetura é um sistema de reservas de hotel, pois é possível ter um serviço independente para gerenciar reservas, outro para lidar com pagamentos e outro para gerenciar inventário de quartos. Cada um desses serviços funciona separadamente, mas colabora com os outros para formar o sistema completo.

APIs de comunicação

Os microserviços comunicam-se entre si usando APIs bem definidas, e isso geralmente é feito por protocolos HTTP/REST ou mensagens assíncronas, como filas de mensagens. Por exemplo, um serviço de carrinho de compras pode se comunicar com o serviço de catálogo de produtos para obter informações sobre os itens adicionados ao carrinho. Essa comunicação é feita através de uma API RESTful que permite ao serviço de carrinho de compras fazer chamadas HTTP ao serviço de catálogo.

Banco de dados descentralizados

Cada microsserviço pode ter seu próprio banco de dados, permitindo uma maior independência e especialização. Isso facilita a escalabilidade e a manutenção, pois cada serviço gerencia seus próprios dados. Um serviço de pagamento pode usar um banco de dados relacional para transações financeiras, enquanto o serviço de catálogo de produtos pode usar um banco de dados NoSQL para gerenciar informações de produtos. Cada banco de dados é otimizado para os requisitos específicos do serviço que atende.

Benefícios dos microsserviços

Escalabilidade granular

Permite escalar individualmente cada serviço conforme necessário, em vez de escalar a aplicação inteira.

Exemplo: se a demanda por processamento de pagamentos aumenta, você pode escalar apenas o serviço de pagamentos, sem precisar escalar outros.

Resiliência

Falhas em um serviço específico geralmente não afetam diretamente outros serviços, aumentando a resiliência do sistema.

Exemplo: se o serviço de recomendações falhar, o restante da aplicação (como a capacidade de navegar e comprar produtos) permanece funcional, minimizando o impacto no usuário final.

Flexibilidade de desenvolvimento

Diferentes equipes podem trabalhar em diferentes serviços simultaneamente, usando as tecnologias e ferramentas que melhor se adaptam às necessidades específicas de cada serviço.

Exemplo: a equipe responsável pelo serviço de envio pode escolher usar uma linguagem de programação otimizada para tarefas de logística, enquanto a equipe de pagamentos pode usar outra linguagem mais adequada para segurança financeira.

Desafios dos microsserviços

Complexidade de gestão

A gestão de múltiplos serviços independentes pode ser complexa. Requer ferramentas e práticas avançadas de gestão e automação.

Exemplo: gerenciar a comunicação e a integração contínua entre dezenas ou centenas de microsserviços pode ser desafiador, exigindo soluções robustas de orquestração e monitoramento.

Comunicação e latência

A comunicação entre serviços pode introduzir latência e aumentar a complexidade da depuração, o que pode impactar o desempenho do sistema.

Exemplo: cada chamada de API entre serviços introduz uma sobrecarga de rede, que pode somar-se rapidamente se tivermos muitas interações entre serviços, impactando a performance geral.

Implantação e monitoramento

Requer uma infraestrutura robusta para implantação e monitoramento dos serviços. Isso inclui ferramentas para automação de deploy, monitoramento de performance e logging.

Exemplo: ferramentas como Kubernetes para orquestração de contêineres e Prometheus para monitoramento se tornam essenciais para gerenciar a complexidade dos microsserviços em produção.

Quando usar microsserviços

A arquitetura de microsserviços é ideal para projetos grandes e complexos, onde a escalabilidade, a flexibilidade e a modularidade são cruciais. Algumas situações em que esta arquitetura pode ser a melhor escolha incluem:

Grandes aplicações: sistemas com muitas funcionalidades diferentes e requisitos de escalabilidade elevados podem se beneficiar da modularidade dos microsserviços.

Exemplo: uma plataforma de streaming como a Netflix, que precisa suportar milhões de usuários e várias funcionalidades, como streaming de vídeo, recomendações personalizadas e gerenciamento de contas.

Equipes grandes e distribuídas: em equipes grandes ou distribuídas, a independência dos microsserviços permite que diferentes equipes trabalhem em paralelo sem interferência.

Exemplo: uma grande empresa de tecnologia com equipes distribuídas globalmente pode desenvolver e manter serviços específicos de forma independente, facilitando a colaboração e a inovação contínua.

Requisitos de alta disponibilidade: aplicações que precisam estar disponíveis 24/7 podem se beneficiar da resiliência dos microsserviços, onde falhas em um serviço não afetam diretamente os outros.

Exemplo: um banco on-line que oferece serviços como transferências bancárias, pagamento de contas e consulta de saldo, precisa garantir que uma falha em um serviço não comprometa toda a plataforma.

Comparação entre microsserviços e monolitos

A comparação entre as arquiteturas monolítica e de microsserviços pode ser realizada considerando diversos aspectos críticos para o desenvolvimento e operação de sistemas de software. Vamos explorar alguns desses aspectos.

Arquitetura

Monolitos: a arquitetura monolítica agrupa todas as funcionalidades de uma aplicação em um único código-base. Isso facilita o desenvolvimento inicial e a implantação, pois há apenas um artefato para gerenciar. No entanto, essa abordagem pode se tornar um gargalo à medida que a aplicação cresce em complexidade.

Microsserviços: a arquitetura de microsserviços divide a aplicação em serviços menores e independentes. Cada serviço pode ser desenvolvido, implantado e escalado de forma independente. Isso promove uma maior modularidade e flexibilidade, permitindo uma evolução mais rápida e eficiente da aplicação.

Escalabilidade

Monolitos: escalar uma aplicação monolítica geralmente envolve a replicação da aplicação inteira. Isso pode ser ineficiente, especialmente se apenas uma parte da aplicação requer mais recursos.

Microsserviços: com microsserviços, é possível escalar apenas os serviços que precisam de mais recursos. Isso permite uma utilização mais eficiente dos recursos e uma escalabilidade mais granular.

Desenvolvimento

Monolitos: o desenvolvimento em uma arquitetura monolítica pode ser mais simples no início, pois todas as dependências estão em um único código-base. No entanto, à medida que a aplicação cresce, o desenvolvimento pode se tornar mais difícil devido à complexidade crescente do código.

Microsserviços: o desenvolvimento em uma arquitetura de microsserviços permite que diferentes equipes trabalhem em serviços diferentes de forma independente. Isso pode acelerar o desenvolvimento e permitir maior especialização, mas também requer uma coordenação cuidadosa para garantir a compatibilidade entre os serviços.

Resiliência

Monolitos: uma falha em um componente de uma aplicação monolítica pode afetar toda a aplicação, levando a um tempo de inatividade significativo.

Microsserviços: em uma arquitetura de microsserviços, uma falha em um serviço geralmente não afeta outros, o que aumenta a resiliência do sistema e minimiza o impacto de falhas individuais.

Complexidade

Monolitos: a complexidade de uma aplicação monolítica está centralizada em um único código-base. Isso pode ser vantajoso no início, mas pode se tornar uma desvantagem à medida que a aplicação cresce.

Microsserviços: a arquitetura de microsserviços distribui a complexidade entre vários serviços independentes. Embora isso possa reduzir a complexidade de cada serviço individual, a gestão e a orquestração de todos os serviços podem se tornar um desafio significativo.

Implantação

Monolitos: a implantação de uma aplicação monolítica é um processo único e coeso. Isso pode simplificar a gestão de releases, mas também significa que qualquer mudança requer a reimplantação de toda a aplicação.

Microsserviços: a implantação de microsserviços é independente para cada serviço. Isso permite uma maior flexibilidade e frequência de releases, mas também requer ferramentas e práticas avançadas para gerenciar o processo de deploy.

Monolitos modulares

Além dos monolitos tradicionais e dos microsserviços, existe uma abordagem intermediária chamada **monolito modular**. Esta abordagem combina elementos de ambas as arquiteturas, oferecendo uma maneira eficaz de estruturar sistemas grandes e complexos.

Os monolitos modulares são arquiteturas que, embora mantenham a aplicação como uma unidade indivisível, enfatizam a modularidade interna. Isso significa que a aplicação é dividida em módulos ou componentes bem definidos, cada um responsável por uma funcionalidade específica.

Características dos monolitos modulares

Separação de preocupações

Cada módulo encapsula uma funcionalidade específica da aplicação, promovendo a separação de preocupações, e isso facilita a manutenção e evolução do sistema. Um sistema de gerenciamento de projetos, por exemplo, pode ter módulos separados para gerenciamento de tarefas, usuários, relatórios e notificações.

Interface clara entre módulos

A comunicação entre módulos é feita por interfaces bem definidas. Isso ajuda a minimizar as dependências e facilita a substituição ou atualização de módulos individuais. Sendo assim, um módulo de relatórios pode se comunicar com o módulo de gerenciamento de tarefas através de uma interface que fornece dados sobre o progresso das tarefas.

Deploy unificado

Apesar da modularidade interna, a aplicação ainda é implantada como uma única unidade. Isso simplifica o processo de deploy, pois não há necessidade de gerenciar múltiplos serviços independentes. Exemplificando, quando uma nova funcionalidade é adicionada ao módulo de notificações, toda a aplicação é reimplantada, mas os outros módulos permanecem inalterados.

Vantagens dos monolitos modulares

Facilidade de manutenção: a modularidade interna facilita a compreensão e manutenção do código, pois cada módulo é responsável por uma parte específica da aplicação.

Evolução gradual: permite a evolução gradual para uma arquitetura de microsserviços, se necessário. Módulos podem ser convertidos em serviços independentes ao longo do tempo.

Desempenho: mantém as vantagens de desempenho dos monolitos tradicionais, com comunicação rápida e direta entre os módulos.

Desvantagens dos monolitos modulares

Complexidade inicial: inicialmente pode ser mais complexo de implementar em comparação aos monolitos tradicionais, devido à necessidade de definir interfaces claras e seguras entre módulos.

Deploy Único: ainda exige a reimplantação da aplicação inteira para mudanças, embora a modularidade interna possa mitigar alguns dos desafios associados.

Quando usar monolitos modulares

A abordagem de monolito modular pode ser ideal para situações em que a simplicidade de um monolito é desejável, mas a aplicação também se beneficiaria da separação interna de preocupações e modularidade.

Exemplos de Uso:

Sistemas de médio porte: aplicações que precisam de certa modularidade, mas ainda podem ser geridas como uma única unidade.

Equipes de tamanho médio: equipes que desejam trabalhar em diferentes partes da aplicação de forma semi-independente, mas sem a complexidade completa de microsserviços.

Evolução para microsserviços: empresas que planejam migrar para microsserviços eventualmente, mas desejam começar com uma arquitetura mais simples e coesa.

MERCADO, CASES E TENDÊNCIAS

Microsserviços em ação: Amazon

A Amazon é um exemplo clássico de uma empresa que adotou a arquitetura de microsserviços para suportar seu vasto e complexo ecossistema de comércio eletrônico. Originalmente construída como uma aplicação monolítica, a Amazon migrou para uma arquitetura de microsserviços para melhorar a escalabilidade e a flexibilidade de sua plataforma.

Benefícios da arquitetura de microsserviços para a Amazon

- **Escalabilidade:** permite que diferentes partes da aplicação, como o sistema de pagamento, catálogo de produtos e gerenciamento de inventário, sejam escaladas independentemente conforme a demanda.
- **Desenvolvimento ágil:** equipes independentes podem trabalhar em diferentes serviços simultaneamente, acelerando a entrega de novas funcionalidades e melhorias.
- **Resiliência:** falhas em um serviço específico, como o serviço de recomendações, não afetam diretamente outros serviços, garantindo maior disponibilidade do sistema.

No entanto, vale destacar que, em uma situação específica, a Amazon Prime Video decidiu voltar para uma arquitetura monolítica. A equipe do Prime Video percebeu que, para sua necessidade específica de alto desempenho e simplicidade de deploy, um monolito poderia oferecer vantagens. Esta decisão foi tomada para reduzir a complexidade e melhorar a eficiência operacional, demonstrando que a escolha arquitetural deve sempre considerar o contexto e os requisitos específicos do sistema.

Monolitos em ação: Stack Overflow

O Stack Overflow é um excelente exemplo de sucesso de uma aplicação monolítica. Ao contrário da tendência crescente de adoção de microsserviços, o Stack

Overflow permaneceu com uma arquitetura monolítica para oferecer uma plataforma robusta, eficiente e de alta performance.

Benefícios da arquitetura monolítica para o Stack Overflow:

- **Simplicidade:** a estrutura coesa facilita o desenvolvimento, testes e manutenção, permitindo que a equipe foque na entrega de valor aos usuários.
- **Desempenho:** a comunicação direta entre componentes resulta em alta performance, crucial para a experiência do usuário em uma plataforma com altíssimo tráfego.
- **Gerenciamento simplificado:** um único código-base é mais fácil de gerenciar, especialmente para equipes menores, garantindo uma integração contínua e sem interrupções.

O Stack Overflow demonstrou que uma arquitetura monolítica bem projetada pode ser altamente eficiente e escalável, desafiando a noção de que os microsserviços são sempre a melhor solução para grandes sistemas.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos dois modelos arquiteturais fundamentais: monolitos e microsserviços. Aprendemos que os monolitos são aplicações construídas como uma única unidade indivisível, com simplicidade e desempenho, mas com desafios de escalabilidade e manutenção. Por outro lado, os microsserviços dividem a aplicação em serviços menores e independentes, oferecendo maior escalabilidade e flexibilidade, mas introduzindo complexidade de gestão e comunicação. Também discutimos a abordagem intermediária dos monolitos modulares, que combinam a simplicidade dos monolitos com a modularidade interna.

Analizamos casos práticos, como o da Amazon, que migrou para microsserviços mas que voltou para monolitos em algumas situações específicas, e o Stack Overflow, que permaneceu bem-sucedido com uma arquitetura monolítica. Entendemos que a escolha da arquitetura deve ser baseada no contexto e nas necessidades específicas do projeto, levando em consideração fatores como tamanho do sistema, requisitos de escalabilidade e estrutura da equipe.

REFERÊNCIAS

BANSAL, N. Case Study: How StackOverflow's Monolith Beats Microservices. **LinkedIn**, 2023. Disponível em: <https://www.linkedin.com/pulse/case-study-how-stackoverflows-monolith-beats-navjot-bansal>. Acesso em: 23 set. 2024.

NEWMAN, S. **Building Microservices. 2nd ed.** [s.l.]: O'Reilly Media, 2021.

PRADEEP, N. Amazon Prime Moved from Microservices to Monolith Except for Scaling. **LinkedIn**, 2023. Disponível em: <https://www.linkedin.com/pulse/amazon-prime-moved-from-microservices-monolith-except-nithin-j>. Acesso em: 23 set. 2024.

RICHARDSON, C. **Microservices Patterns: With examples in Java.** [s.l.]: Manning Publications, 2018.

PALAVRAS-CHAVE

Arquiteto, Arquitetura de Software, Modularização, Microserviços, Monolito, Escalabilidade, Complexidade.

EXEMPLO

POS TECH